

TorchDCM 项目计划书

基于 PyTorch 的离散选择模型估计库

Biogeme/Apollo 模型覆盖 + statsmodels 级推断 + scikit-learn 风格 API +
C++/CUDA 加速

目标定位：构建一个面向交通、市场营销、环境经济学和城市数据科学的现代 discrete choice model estimation Python 库。它应同时具备传统 choice modelling 软件的严肃统计推断能力、PyTorch 的自动微分/GPU 工程能力，以及 scikit-learn 生态的易用接口。

版本：规划草案 v1.0

日期：2026-07-03

目录

1 一句话定位	3
2 现有软件格局与差异化机会	3
3 设计原则	3
3.1 Estimation-first, 而不是 prediction-first	3
3.2 PyTorch backend, 但默认 float64	4
3.3 先纯 PyTorch, 再 C++/CUDA	4
3.4 双层 API	4
4 包结构建议	5
5 核心数据结构: Ragged choice-set representation	6
6 模型覆盖路线	7
7 API 设计	8
7.1 Programmatic DSL	8
7.2 Formula API	8
7.3 Mixed logit API	8
7.4 scikit-learn wrapper	9
8 Estimation engine	9
9 Inference: 做到 statsmodels 级别	10
9.1 基础输出	10
9.2 Classical Hessian covariance	10
9.3 OPG/BHHH covariance	11
9.4 Robust sandwich covariance	11
9.5 Cluster robust covariance	11
9.6 Delta method: WTP/VOT/elasticity	11
10 计算创新与加速设计	12
10.1 创新 1: Ragged choice-set fused logsumexp kernel	12
10.2 创新 2: Panel mixed logit streaming logmeanexp kernel	12
10.3 创新 3: per-observation score 的 vectorized AD + analytic fallback	12
10.4 创新 4: Specification compiler	13
11 Result object 设计	13
12 MVP: v0.1 必须做什么	14
13 开发阶段建议	15

14 测试、验证与 benchmark	15
14.1 数值正确性	15
14.2 对标数据与软件	16
14.3 Benchmark 指标	16
15 开源发布和论文角度	16
16 关键风险与决策	17
17 建议的 GitHub issue 初始列表	17
18 结论	18
参考资料	19

1 一句话定位

torchdcm 的核心定位是: **A PyTorch-based, econometrics-grade, differentiable discrete choice modelling library with Biogeme/Apollo-level model coverage, statsmodels-level inference, and scikit-learn-style usability.**

这不是“再写一个 MNL 包”。更准确的目标是把三个生态的优势合并:

- **Biogeme/Apollo 式模型能力**: MNL、NL、CNL、mixed logit、latent class、hybrid choice/ICLV、MDCEV 等;
- **statsmodels 式统计推断**: standard errors、robust covariance、cluster covariance、Wald/LR/score tests、置信区间、delta-method WTP/VOT;
- **PyTorch/ML 式工程体验**: autograd、GPU、mini-batch、torch.compile、自定义 C++/CUDA 算子、神经网络 utility;
- **scikit-learn 式 API**: fit、predict、predict_proba、get_params、set_params、cross-validation、pipeline 兼容。

2 现有软件格局与差异化机会

Biogeme 官方文档将其定义为一个开源 Python 包, 主要用于 maximum likelihood estimation, 并特别强调 discrete choice models **[R1]**。Apollo 是 R 生态中非常灵活的 choice modelling 包, 官方介绍强调 continuous/discrete random heterogeneity、hybrid model、classical/Bayesian estimation、multiple discrete-continuous models、多线程和丰富的 pre/post-estimation routines **[R2]**。另一方面, torch-choice、xlogit、Choice-Learn 代表了 Python 生态中更偏大规模/GPU/ML 的方向: torch-choice 以 PyTorch 实现 conditional logit 和 nested logit, 并强调大规模 choice records **[R3]**; xlogit 聚焦 GPU-accelerated mixed logit **[R4]**; Choice-Learn 则强调大规模 choice data、RAM 友好和 operational contexts **[R5]**。

差异化主张: torchdcm 应该主打 “**estimation + inference + acceleration**”, 而不是单纯 prediction。很多 ML 化 choice 库可以预测, 但 DCM 用户真正需要可发表论文、可做政策仿真、可解释的统计结果。

3 设计原则

3.1 Estimation-first, 而不是 prediction-first

DCM 用户关心的不是只有 predict_proba, 还包括:

- 参数显著性、置信区间、robust/cluster standard errors;
- WTP/VOT、elasticity、logsum、consumer surplus、market share simulation;

软件/库	强项	给 torchdcm 的机会
Biogeme	Python 生态中传统 DCM 的重要平台; MLE; 大量示例; 支持高级模型。	API 和 ML/GPU 工程体验仍有提升空间; 可做 PyTorch-native + modern inference。
Apollo	R 生态功能覆盖极广; random heterogeneity, hybrid choice, Bayesian, MDCEV 等。	Python 生态缺少 Apollo 级覆盖且 ML-native 的统一实现。
torch-choice	PyTorch、GPU、大规模、conditional/nested logit。	模型覆盖较窄; 可补充 mixed logit, CNL, ICLV, MDCEV 和完整推断。
xlogit	GPU 加速 mixed logit, 速度优势明显。	可把 GPU mixed logit 融入更完整的模型、推断和 API 体系。
Choice-Learn	大规模 choice modelling, TensorFlow、可部署、assortment/pricing 场景。	交通/经济学用户所需的 Biogeme/Apollo 型统计输出和高级模型仍有空间。
statsmodels	离散因变量模型、summary、robust covariance、marginal effects 等统计体验。	不是专门的 DCM 平台; 缺少 NL/CNL/mixed logit/choice-set sampling 等功能。

- 反事实政策模拟和 welfare analysis;
- 与 Biogeme/Apollo/statsmodels 结果可复现和可对齐。

3.2 PyTorch backend, 但默认 float64

MLE、Hessian、Fisher information、mixed logit simulation 对数值精度敏感。因此默认应为:

```
dtype = torch.float64
device = "cpu"
```

用户可以显式切换到:

```
dtype = torch.float64
device = "cuda"
```

不建议默认 float32, 否则在 Hessian inversion、robust covariance、simulated likelihood 中容易出现不稳定。

3.3 先纯 PyTorch, 再 C++/CUDA

第一阶段应先用纯 PyTorch 写 reference backend, 确保 likelihood、gradient、Hessian 和 inference 完全正确。之后再针对最慢的 segment reduction、panel mixed logit、per-observation score 写 optional C++/CUDA kernel。PyTorch 官方支持自定义 C++/CUDA operator 并接入 autograd 等子系统 [R8]。

3.4 双层 API

- **Econometric DSL:** 面向 Biogeme/Apollo 用户, 允许显式定义 utility、random coefficients、nests、latent variables;

- **sklearn estimator API**: 面向 ML 用户, 支持 `fit`、`predict`、`score`、`get_params`、`set_params`, 并可接入 `GridSearchCV` 和 `pipelines`。scikit-learn 官方开发指南要求估计器具备统一参数接口 [R7]。

4 包结构建议

```
torchdcm/  
  data/  
    choice_dataset.py  
    converters.py  
    samplers.py  
  spec/  
    parameters.py  
    expressions.py  
    utility.py  
    nests.py  
    latent.py  
    mdcev.py  
  models/  
    base.py  
    mnl.py  
    conditional_logit.py  
    nested_logit.py  
    cross_nested_logit.py  
    mixed_logit.py  
    latent_class.py  
    hybrid_choice.py  
    ordered.py  
    mdcev.py  
    neural_choice.py  
  estimation/  
    objective.py  
    optimizers.py  
    constraints.py  
    draws.py  
    em.py  
    bayes.py  
  inference/  
    covariance.py  
    robust.py  
    cluster.py  
    hessian.py  
    scores.py  
    hypothesis_tests.py  
    marginal_effects.py  
    elasticities.py  
    wtp.py  
    simulation.py  
  kernels/
```

```

ragged_logsumexp.cpp
ragged_logsumexp_cuda.cu
mixed_logit_panel.cpp
mixed_logit_panel_cuda.cu
results/
  result.py
  summary.py
  tables.py
  latex.py
sklearn/
  estimators.py
datasets/
  swissmetro.py
  modecanada.py
  heating.py
benchmarks/
  compare_biogeme.py
  compare_xlogit.py
  speed_tests.py

```

核心对象建议固定为: ChoiceDataset、Beta/RandomBeta、UtilitySpec、ChoiceModel、ChoiceResults。

5 核心数据结构: Ragged choice-set representation

不要把内部结构设计成单一 pandas DataFrame, 也不要强依赖 padded $N \times J_{\max}$ matrix。最通用的 DCM 数据结构应支持 variable choice set、availability、panel、weights、sampled alternatives、alternative-specific variables 和 individual-specific variables。

统一表示为:

$$\mathcal{D} = \{(n, t, C_{nt}, y_{nt}, X_{ntj}, Z_n, A_{ntj}, w_{nt})\},$$

其中 n 是 decision maker, t 是 choice occasion, C_{nt} 是可选 alternative 集合, y_{nt} 是被选 alternative, X_{ntj} 是 alternative-specific variables, Z_n 是 individual-specific variables, A_{ntj} 是 availability, w_{nt} 是 observation weight。

内部建议使用 offset arrays:

```

ChoiceDataset(
  obs_ptr: Tensor[N + 1], # observation -> row segment
  alt_id: Tensor[M], # long-format alternative id
  chosen_row: Tensor[N], # chosen row index for each obs
  x_alt: Dict[str, Tensor[M]], # alternative-specific variables
  x_obs: Dict[str, Tensor[N]], # observation-specific variables
  x_ind: Dict[str, Tensor[I]], # individual-specific variables
  obs_to_ind: Tensor[N],
  weights: Tensor[N],
  availability: Tensor[M],
)

```

为什么这个设计重要：这会直接决定后续 C++/CUDA kernel 是否能自然支持 variable choice set。MNL 的 logsumexp、NL 的 nest-level reduction、mixed logit 的 panel likelihood 都可以基于 segment reduction 统一实现。

6 模型覆盖路线

建议按版本逐步推进，不要在 v0.1 试图复刻 Apollo 全部功能。

版本	模型	核心功能
v0.1	MNL / conditional logit	long/wide converter、availability、weights、generic/alternative-specific coefficients、MLE、summary、classical/robust/cluster SE。
v0.2	Nested logit / cross-nested logit	nest constraints、inclusive value、logsum、elasticity、WTP、LR/Wald/score tests。
v0.3	Mixed logit	random coefficients、normal/lognormal/uniform/triangular/truncated normal、Sobol/Halton/antithetic draws、panel likelihood、individual posterior parameters。
v0.4	Latent class / finite mixture	class allocation model、EM、direct MLE、posterior class probability、mixed latent class。
v0.5	Hybrid choice / ICLV	latent variables、structural equations、measurement equations、continuous/ordered indicators。
v0.6	Ordered logit/probit, exploded logit, best-worst	rating/ranking/best-worst 数据结构和 likelihood。
v0.7	MDCEV / discrete-continuous choice	outside good、alpha-gamma profile、budget constraints。
v0.8	Neural DCM	TasteNet-style heterogeneity、RUMnet/Res-Logit/residual utility、embedding-based utility。

优先路径建议为：

MNL → NL/CNL → Mixed Logit → Latent Class → ICLV.

7 API 设计

7.1 Programmatic DSL

```
from torchdcm import ChoiceDataset, Beta, UtilitySpec, MultinomialLogit

B_TIME = Beta("B_TIME", init=-0.01)
B_COST = Beta("B_COST", init=-0.10)
ASC_CAR = Beta("ASC_CAR", init=0.0)
ASC_TRAIN = Beta("ASC_TRAIN", init=0.0)
ASC_SM = Beta("ASC_SM", init=0.0, fixed=True)

spec = UtilitySpec()
spec.utility("car", ASC_CAR + B_TIME * "time_car" + B_COST * "cost_car")
spec.utility("train", ASC_TRAIN + B_TIME * "time_train" + B_COST * "cost_train")
spec.utility("sm", ASC_SM + B_TIME * "time_sm" + B_COST * "cost_sm")

model = MultinomialLogit(spec, dtype="float64")
res = model.fit(data, cov_type="cluster", groups="person_id")
res.summary()
```

7.2 Formula API

```
model = MultinomialLogit.from_formula(
    utilities={
        "car": "ASC_CAR + B_TIME * time + B_COST * cost",
        "train": "ASC_TRAIN + B_TIME * time + B_COST * cost",
        "sm": "B_TIME * time + B_COST * cost",
    },
    alt_id="alt",
    obs_id="choice_id",
    choice="chosen",
)
res = model.fit(df_long)
```

7.3 Mixed logit API

```
from torchdcm import RandomBeta, MixedLogit

B_TIME = RandomBeta("B_TIME", dist="lognormal", sign="negative",
                    mean_init=-3.0, sd_init=0.5)
B_COST = RandomBeta("B_COST", dist="normal",
                    mean_init=-0.1, sd_init=0.02)

model = MixedLogit(
    spec,
    random_coefficients=[B_TIME, B_COST],
    panel_id="person_id",
```

```

n_draws=512,
draw_type="sobol",
antithetic=True,
device="cuda",
)
res = model.fit(data)
res.summary()
res.individual_parameters()

```

7.4 scikit-learn wrapper

```

from torchdcm.sklearn import TorchDCMClassifier

clf = TorchDCMClassifier(
    model="mnl",
    spec=spec,
    optimizer="lbfgs",
    cov_type="robust",
    score_metric="log_likelihood",
    device="cpu",
)

clf.fit(df_long)
proba = clf.predict_proba(df_test)
pred = clf.predict(df_test)
score = clf.score(df_test)

```

8 Estimation engine

所有模型统一成 abstract likelihood:

$$\ell(\theta) = \sum_{n=1}^N w_n \log P(y_n | C_n, X_n, Z_n; \theta).$$

MNL 的 choice probability:

$$P_{ni} = \frac{\exp(V_{ni})}{\sum_{j \in C_n} \exp(V_{nj})}.$$

Mixed logit:

$$P_{ni} = \int \frac{\exp(V_{ni}(\beta))}{\sum_{j \in C_n} \exp(V_{nj}(\beta))} f(\beta | \theta) d\beta,$$

其 simulated MLE 形式为:

$$\hat{P}_{ni} = \frac{1}{R} \sum_{r=1}^R P_{ni}(\beta^{(r)}).$$

Panel mixed logit:

$$L_n(\theta) = \int \prod_{t=1}^{T_n} P_{nty_{nt}}(\beta) f(\beta | \theta) d\beta,$$

$$\log \hat{L}_n = \log \left[\frac{1}{R} \sum_{r=1}^R \exp \left(\sum_{t=1}^{T_n} \log P_{nty_{nt}}(\beta^{(r)}) \right) \right].$$

必须使用 stable log-sum-exp:

$$\log \frac{1}{R} \sum_r \exp(a_r) = m + \log \sum_r \exp(a_r - m) - \log R, \quad m = \max_r a_r.$$

优化器建议提供三类:

- `lbfgs`: deterministic MLE 默认选择;
- `adam_lbfgs`: Adam warm start + LBFGS refinement;
- `scipy`: SciPy BFGS/L-BFGS-B/trust-constr wrapper, 便于边界约束和调试。

约束参数全部通过 unconstrained transform 实现:

参数	约束	内部参数化
标准差 σ	$\sigma > 0$	$\sigma = \text{softplus}(\rho)$
Nest 参数 λ	$0 < \lambda \leq 1$	$\lambda = \text{sigmoid}(\rho)$ 或 scaled sigmoid
Class probability	和为 1	$\pi = \text{softmax}(\alpha)$
CNL allocation α_{jm}	非负、对 nest 求和为 1	对每个 alternative 做 softmax
负 lognormal time/-cost	符号约束	$-\exp(\mu + \sigma\epsilon)$

9 Inference: 做到 statsmodels 级别

statsmodels 的离散模型模块支持 Logit、Probit、MNLogit 等有限/定性因变量模型 **[R6]**, 而其 robust covariance 接口会让推断统计量基于指定 covariance matrix 重新计算 **[R7]**。torchdcm 应该在 DCM 模型上复制这种体验。

9.1 基础输出

`res.summary()` 至少包括: parameter、estimate、std err、z/t value、p-value、confidence interval、fixed flag、covariance type。模型层面输出:

$$\ell(\hat{\theta}), \quad \ell(0), \quad \rho^2, \quad \bar{\rho}^2, \quad AIC, \quad BIC, \quad N.$$

9.2 Classical Hessian covariance

$$\widehat{\text{Var}}(\hat{\theta}) = \left[-\nabla^2 \ell(\hat{\theta}) \right]^{-1}.$$

9.3 OPG/BHHH covariance

Per-observation score:

$$s_n(\hat{\theta}) = \nabla_{\theta} \ell_n(\hat{\theta}).$$

OPG matrix:

$$B = \sum_{n=1}^N s_n s_n^{\top}.$$

9.4 Robust sandwich covariance

$$\hat{V}_{\text{robust}} = H^{-1} B H^{-1}, \quad H = - \sum_n \nabla_{\theta}^2 \ell_n(\hat{\theta}).$$

9.5 Cluster robust covariance

对于 panel data 或 repeated SP experiments, cluster robust 必须是一等公民:

$$S_g = \sum_{n \in g} s_n,$$

$$\hat{V}_{\text{cluster}} = H^{-1} \left(\sum_g S_g S_g^{\top} \right) H^{-1}.$$

API:

```
res.cov_params(cov_type="classic")
res.cov_params(cov_type="robust")
res.cov_params(cov_type="cluster", groups=data.person_id)
res.get_robustcov_results(cov_type="cluster", groups=data.person_id)
```

9.6 Delta method: WTP/VOT/elasticity

WTP 常见形式:

$$\text{WTP}_k = - \frac{\beta_k}{\beta_{\text{cost}}}.$$

Delta method:

$$\widehat{\text{Var}}(g(\hat{\theta})) = G \hat{V} G^{\top}, \quad G = \nabla_{\theta} g(\hat{\theta}).$$

API:

```
res.wtp(attribute="time", cost="cost")
res.elasticity(variable="cost", alternative="car")
res.marginal_effects(variable="income", at="mean")
```

10 计算创新与加速设计

10.1 创新 1: Ragged choice-set fused logsumexp kernel

MNL likelihood 对每个 observation:

$$\ell_n = V_{ny_n} - \log \sum_{j \in C_n} \exp(V_{nj}).$$

C++/CUDA kernel 直接基于 obs_ptr 做 segment reduction:

```
for each observation n:
    start = obs_ptr[n]
    end = obs_ptr[n + 1]
    m = max(V[start:end])
    denom = sum(exp(V[start:end] - m))
    ll[n] = V[chosen_row[n]] - m - log(denom)
```

Backward 解析式:

$$\frac{\partial \ell_n}{\partial V_{nj}} = \mathbf{1}(j = y_n) - P_{nj}.$$

这个 kernel 的优势: 不需要 padding、不需要 materialize probability matrix、原生支持 variable choice set、CPU 可 OpenMP 并行、CUDA 可按 segment 做 block-level reduction。

10.2 创新 2: Panel mixed logit streaming logmeanexp kernel

Naive mixed logit 容易产生 $N \times R \times T \times J$ 张量, 内存压力巨大。Streaming 版本只维护:

$$a_{nr} = \sum_t \log P_{nty_{nt}}(\beta^{(r)}),$$

$$\log L_n = \log \frac{1}{R} \sum_r \exp(a_{nr}).$$

Backward 可使用 draw responsibility:

$$\omega_{nr} = \frac{\exp(a_{nr})}{\sum_{r'} \exp(a_{nr'})},$$

$$\nabla_{\theta} \log L_n = \sum_r \omega_{nr} \nabla_{\theta} a_{nr}.$$

这会显著降低 panel mixed logit 的内存占用, 是 torchdcm 区别于 naive PyTorch 实现的关键。

10.3 创新 3: per-observation score 的 vectorized AD + analytic fallback

Robust/cluster SE 需要每个 observation 的 score。纯 Python 循环逐个求梯度会非常慢。torchdcm 应支持:

```
score_method="autograd_vmap"
```

```
score_method="analytic_kernel"
```

其中 `autograd_vmap` 使用 PyTorch `torch.func.jacrev/vmap`; PyTorch 文档提供了 `torch.func.jacrev` 作为 Jacobian 计算接口 **[R9]**。对 MNL/NL/mixed logit 的常用情形则提供 `analytic score fallback`。

10.4 创新 4: Specification compiler

不要每次 likelihood evaluation 都重新解析公式。应把 symbolic specification 编译为 execution plan:

```
compiled = spec.compile(data)
```

内部包含 parameter vector、term list、coefficient index、data tensor pointer、alternative mask、constraint transform、utility computation graph。对 linear-in-parameters utility:

$$V = X\beta,$$

可以预编译 block/sparse design matrix。对 repeated SP design 或 semi-aggregate data, 可缓存相同 scenario 的 utility terms。

11 Result object 设计

`fit()` 返回 `ChoiceResults`, 而不是只修改 `model` 本身。

```
res = model.fit(data)

res.params
res.bse
res.tvalues
res.pvalues
res.conf_int()
res.cov_params()
res.summary()
res.summary_latex()
res.predict_proba(data)
res.predict(data)
res.elasticity(...)
res.wtp(...)
res.simulate_counterfactual(...)
```

内部结构建议:

```
class ChoiceResults:
    model
    params
    raw_params
    loglike
    null_loglike
    gradient_norm
```

```
hessian
cov
cov_type
n_obs
n_individuals
n_params
convergence_status
optimizer_result
```

12 MVP: v0.1 必须做什么

v0.1 的目标应非常具体:

v0.1 目标: 复现传统 MNL/conditional logit 的估计和统计推断, 同时提供 PyTorch auto-grad、GPU compatibility、ragged choice sets 和干净的 sklearn-style API。

v0.1 checklist:

1. `ChoiceDataset.from_long()`;
2. `ChoiceDataset.from_wide()`;
3. MNL / conditional logit;
4. availability;
5. observation weights;
6. generic coefficient;
7. alternative-specific coefficient;
8. fixed parameter;
9. constrained parameter transform;
10. log-likelihood / gradient / Hessian;
11. classical covariance;
12. robust sandwich covariance;
13. cluster robust covariance;
14. `summary()`;
15. `predict_proba()`;
16. `predict()`;
17. `elasticity()`;
18. `wtp()`;

19. Swissmetro / ModeCanada example;
20. finite-difference gradient tests;
21. benchmark against Biogeme/statsmodels where applicable.

13 开发阶段建议

阶段	目标	产出
Phase 0	技术原型	纯 PyTorch MNL likelihood; long-format converter; gradient check; 简单 summary。
Phase 1	v0.1 core	完整 MNL/conditional logit、availability、weights、standard errors、robust/cluster co-variance、Swissmetro notebook。
Phase 2	API 稳定	Programmatic DSL、formula API、sklearn wrapper、ChoiceResults、LaTeX/HTML summary。
Phase 3	C++/CUDA kernel	Ragged logsumexp custom op; forward/backward 对齐 PyTorch reference; CPU/GPU benchmark。
Phase 4	NL/CNL	Nested logit、cross-nested logit、nest constraints、inclusive value、WTP/elasticity。
Phase 5	Mixed logit	random coefficients、Sobol/Halton draws、panel likelihood、streaming logmeanexp kernel、posterior individual parameters。
Phase 6	高级模型	Latent class、ICLV、MDCEV、neural DCM, 形成 software paper。

14 测试、验证与 benchmark

14.1 数值正确性

每个模型都应做：

- probability check: $\sum_{j \in C_n} P_{nj} = 1$;
- finite-difference gradient check:

$$\frac{\ell(\theta + \epsilon e_k) - \ell(\theta - \epsilon e_k)}{2\epsilon} \approx \nabla_k \ell(\theta);$$

- Hessian symmetry check: $H \approx H^\top$;

- covariance positive semi-definiteness diagnostics;
- fixed-parameter and constrained-parameter tests;
- CPU/GPU deterministic reproducibility tests when common random numbers are enabled。

14.2 对标数据与软件

数据	模型	对标对象
Swissmetro	MNL / NL / mixed logit	Biogeme / Apollo
ModeCanada	MNL / NL	Biogeme / mlogit / torch-choice
Electricity choice	mixed logit	Apollo / xlogit
Synthetic data	latent class / ICLV	真值已知, 自测恢复能力
Large synthetic	GPU speed	torch-choice / xlogit / pure PyTorch reference

14.3 Benchmark 指标

- coefficient difference relative to reference;
- log-likelihood difference at optimum;
- standard error difference;
- runtime and peak memory;
- gradient evaluations and convergence rate;
- mixed logit 中随 N, R, T, J 增长的 scaling curve;
- robust/cluster SE 的稳定性和与 statsmodels 风格接口的一致性。

15 开源发布和论文角度

如果把这个库写成 software paper, 建议贡献点为:

1. A unified PyTorch framework for discrete choice estimation and inference;
2. Econometrics-grade post-estimation tools: robust covariance, cluster covariance, delta-method WTP, elasticity, hypothesis tests;
3. Ragged choice-set representation for variable availability and large-scale choice data;
4. Fused C++/CUDA kernels for logit likelihood and panel mixed logit simulated likelihood;
5. Dual API design: Biogeme/Apollo-style DSL and scikit-learn-compatible estimator interface。

可能的软件论文标题:

- **TorchDCM: A Differentiable and Econometrics-Grade PyTorch Library for Discrete Choice Modeling**
- **PyChoiceTorch: Scalable Discrete Choice Estimation with Automatic Differentiation and Robust Inference**

16 关键风险与决策

风险	表现	建议
模型覆盖过早膨胀	ICLV/MDCEV/mixed logit 同时做, 导致 v0.1 不稳定。	v0.1 只做 MNL/conditional logit + inference; 先把可信度做出来。
只追求 GPU 速度	估计结果和标准误不可靠, econometric 用户不信任。	每个加速 kernel 都必须有 PyTorch reference 和 numerical tests。
API 太像深度学习	传统 DCM 用户上手困难。	保留 Biogeme/Apollo 式 DSL 和公式接口。
API 太像传统软件	ML 用户无法接入 pipeline/cross-validation/GPU。	提供 sklearn wrapper 和 PyTorch-native nn.Module。
Hessian 不稳定	near-collinearity 或 weak identification 导致 covariance 爆炸。	提供 diagnostics、regularization、profile likelihood、robust covariance warning。
CUDA kernel 维护成本高	PyTorch 版本变化和平台差异。	kernel 设为 optional backend; pure PyTorch 永远可用。

17 建议的 GitHub issue 初始列表

1. Define ChoiceDataset internal schema and invariants。
2. Implement from_long converter with availability and weights。
3. Implement from_wide converter for common mode-choice datasets。
4. Implement parameter registry: fixed, free, constrained。
5. Implement MNL utility compiler for linear-in-parameters models。
6. Implement ragged MNL likelihood in pure PyTorch。

7. Add probability normalization and shape tests。
8. Add finite-difference gradient tests。
9. Add Hessian computation and classical covariance。
10. Add per-observation scores and OPG covariance。
11. Add robust sandwich covariance。
12. Add cluster robust covariance。
13. Build `ChoiceResults.summary()` table。
14. Add `predict_proba` and `predict`。
15. Add WTP/VOT with delta method。
16. Add direct elasticity and arc elasticity。
17. Add Swissmetro tutorial。
18. Add ModeCanada tutorial。
19. Compare MNL estimates against Biogeme/statsmodels reference。
20. Build sklearn wrapper and estimator compliance tests。
21. Implement formula parser。
22. Implement C++ CPU ragged logsumexp kernel。
23. Implement CUDA ragged logsumexp kernel。
24. Add kernel forward/backward parity tests。
25. Add benchmark scripts and CI smoke tests。

18 结论

`torchdcm` 最值得做的地方不在于“又实现一个 logit”，而在于把 **现代 PyTorch 工程** 和 **严肃 choice modelling inference** 合在一起。当前 Python 生态中，传统 DCM 软件往往工程体验不够现代，而 ML/GPU 化库往往统计推断和高级模型覆盖不够完整。`torchdcm` 的最佳切入点是：

先用 **v0.1** 把 **MNL/conditional logit + statsmodels 级 inference** 做到极稳，再通过 **ragged choice-set kernel** 和 **panel mixed logit streaming kernel** 展示计算创新，最后逐步扩展到 **NL/CNL、mixed logit、latent class** 和 **ICLV**。

参考资料

- [R1] Biogeme official documentation. <https://biogeme.epfl.ch/sphinx/index.html>
- [R2] Apollo official website. <https://www.apollochoicemodelling.com/>
- [R3] torch-choice documentation. <https://gsbdbi.github.io/torch-choice/>
- [R4] xlogit documentation. <https://xlogit.readthedocs.io/>
- [R5] Choice-Learn documentation. <https://artefactory.github.io/choice-learn/>
- [R6] statsmodels discrete models documentation. <https://www.statsmodels.org/stable/discretemod.html>
- [R7] statsmodels robust covariance documentation. https://www.statsmodels.org/devel/generated/statsmodels.regression.linear_model.OLSResults.get_robustcov_results.html
- [R8] scikit-learn developer guide: developing estimators. <https://scikit-learn.org/stable/developers/develop.html>
- [R9] PyTorch custom C++ and CUDA operators. https://docs.pytorch.org/tutorials/advanced/cpp_custom_ops.html
- [R10] PyTorch torch.func.jacrev documentation. <https://docs.pytorch.org/docs/stable/generated/torch.func.jacrev.html>